

PROGRAM 8:-

AIM

To implement the Depth First Search (DFS) algorithm to traverse or search all the nodes of a graph by exploring as far as possible along each branch before backtracking.

ALGORITHM

1. Start the program.
2. Create a graph using vertices and edges.
3. Select the starting vertex.
4. Mark the starting vertex as visited.
5. Display the current vertex.
6. Visit an unvisited adjacent vertex recursively.
7. Continue until there are no unvisited adjacent vertices.
8. Backtrack to the previous vertex and continue searching.
9. Repeat until all reachable vertices are visited.
10. Stop the program.

PROGRAM

```
def dfs(graph, vertex, visited):  
    visited.add(vertex)  
    print(vertex, end=" ")  
    for neighbour in graph[vertex]:  
        if neighbour not in visited:  
            dfs(graph, neighbour, visited)  
  
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': [],
```

```
'E': [],  
'F': []  
}  
visited = set()  
print("DFS Traversal:")  
dfs(graph, 'A', visited)
```

OUTPUT

```
DFS Traversal:  
A B D E C F
```

RESULT

The Depth First Search (DFS) algorithm was successfully implemented, and the graph was traversed by exploring each branch deeply before backtracking.